

DEEP LEARNING, TOLD SIMPLY

Book 1 of 5

The Machine That Learned to Listen

Nobody can write the program that hears its own name. So we grow one from examples instead. This is the whole idea of deep learning — told as a story.

Inside: what learning really means · deep learning, defined without jargon · how it works (the learning loop) · the four ingredients of every deep learning project · why deep learning, and why now · supervised learning and the questions it answers

“Hey Nova!”



+0.12 -0.31 +0.07
-0.44 +0.28 +0.02
+0.19 -0.05 ...

Deep Learning (DNN) · a five-book story

Prof. Saurabh

Before you start

This is not a textbook. It is a story in five parts, and this is part one. Read it from the beginning, at your own pace, with a pen nearby. Every big idea comes with a picture, an everyday example, and a small thing to think about. You need no maths beyond school algebra, and no code yet — only curiosity and about an hour.

The boxes you will meet

📖 STORY

Asha and her little robot Nova. Their problems are our problems.

✓ OUR ANSWER

Our take on the question above. Compare it with yours.

💡 BIG IDEA

The one thing to take away. If you remember nothing else, remember this.

🧪 TRY IT YOURSELF

A small experiment with paper, a phone, or your own voice. No code needed.

🔗 THINK OF IT LIKE...

An everyday picture for a new idea. This is how we will understand everything.

⚠️ CAREFUL

A place where many people get confused. We slow down here.

💭 STOP AND THINK

A question for you. Pause and try it before reading on — the trying is where learning happens.

✨ SPARK

A piece of history or a surprising fact, to make you wonder.

The map of all five books

Each book solves one problem and leaves one new problem open. Together they build a real neural network from nothing.



- 1 The Machine That Learned to Listen.** Why some programs cannot be written by hand, what "learning" means for a machine, and the four ingredients of every deep learning project. (*this book*)
- 2 One Tiny Brain Cell.** The smallest possible learner: one artificial neuron. We teach it AND, OR and NOT by hand — then give it a puzzle it cannot solve.
- 3 Guessing "How Much".** One neuron predicts a car's fuel mileage. The loss function, properly, and how to walk downhill: gradient descent.
- 4 Guessing "Which One".** One neuron answers yes/no (is this tumour harmful?); a row of neurons answers "which of three flowers?". Sigmoid, softmax, surprise.
- 5 A Real Brain.** Neurons in layers. Forward pass, backward pass, and the puzzle from Book 2 finally falls. Nova, at last, can hear its name.

1

A job nobody can code by hand

Some things are easy to do and impossible to explain. Hearing your own name is one of them.

STORY

Asha is a first-year student. Over the summer she built a small speaker-robot and named it **Nova**. She wants Nova to do exactly one thing. When someone says “**Hey Nova**”, Nova should wake up and light its eyes. When people say anything else, Nova should stay asleep.

Asha thinks: “*Easy. That is one if-statement.*”



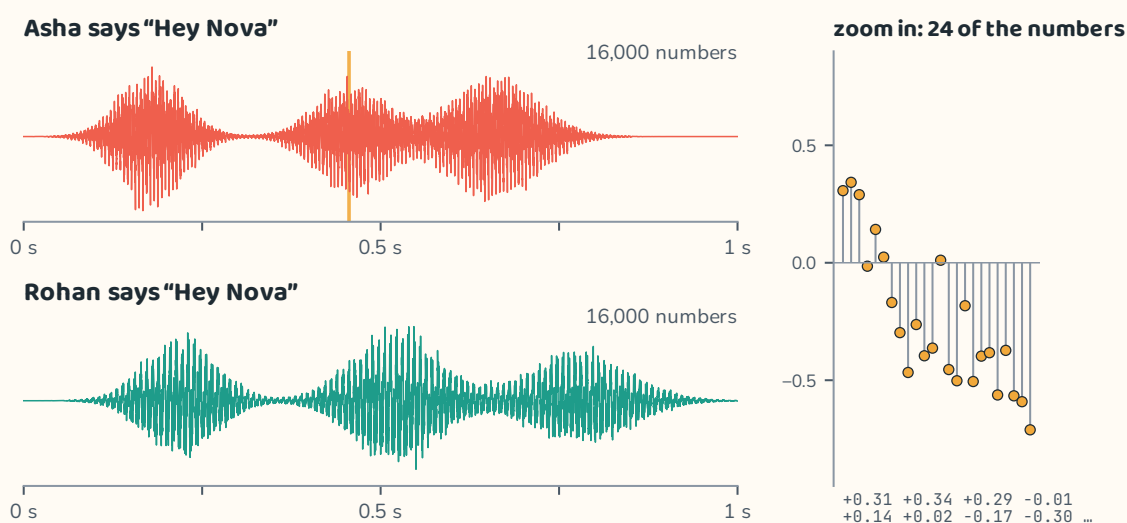
The job. Wake up for “Hey Nova”. Stay asleep for everything else — including things that sound almost the same.

What does Nova actually hear?

Before writing the if-statement, Asha asks a simple question: *what does the computer get when someone speaks?*

Not words. A microphone cannot hear words. It only feels the air pushing on a tiny sheet of plastic. Sixteen thousand times every second, it writes down *how hard* the push is. That is all.

So one second of sound is a list of **16,000 numbers**. No letters. No syllables. Just numbers, one after another, like this: +0.12, -0.31, +0.07, -0.44, ...



Two people, same words, different numbers. Asha’s “Hey Nova” and Rohan’s “Hey Nova” are both lists of 16,000 numbers. On the right we zoom in on 24 of Asha’s numbers. This list of numbers is the *only* thing Nova ever receives.

💡 STOP AND THINK

Look at the two waveforms above. Both are “Hey Nova”. Try to write a rule that says **yes** to both of them and **no** to “Hey Nora”, to a door slam, and to a song on the radio. Take two minutes. Really try.

✓ OUR ANSWER

We could not do it either. The two lists are completely different numbers. A deeper voice, a slower speed, a cold, a fan in the background, a cheap phone speaker — each one gives a brand-new list. There are more ways to say “Hey Nova” than there are seconds in your whole life. And “Hey Nora” gives numbers that look almost the same as “Hey Nova”, yet it must get the *opposite* answer.

Asha tries anyway

Attempt	Asha's rule	What happened
1	“If the sound gets loud in the middle, wake up.”	Nova woke up every time a door closed.
2	“If there are three bursts of sound — Hey · No · va — wake up.”	“Ba-na-na” woke Nova. And Rohan says it fast, as two bursts, so Nova ignored him.
3	Four hundred rules, written over two weeks.	Nova now wakes up for Asha, in her room, when it is quiet. For nobody else. Asha gives up on rules.

⚠️ CAREFUL — THIS IS THE REAL PROBLEM

The problem is *not* that the computer is too slow or too small. The problem is that **we ourselves do not know the rule**. You can recognise your mother's voice in a crowded room in half a second. Can you write down how you do it? No. We know more than we can tell. (Philosophers call this Polanyi's paradox. Now you know its name, but you already knew the feeling.)

💡 BIG IDEA — FROM TELLING TO SHOWING

If you cannot write the program, don't. Collect **examples** instead — many clips of “Hey Nova” and many clips of other sounds — and build a machine that finds the rule by itself.

We stop *telling* the computer what to do. We start *showing* it. This flip, from telling to showing, is the whole idea of **machine learning**. **Deep learning** is one very powerful way of doing it, and this course is about that way.

✨ SPARK

Your phone does this job right now. A tiny chip, using less power than a night-light, listens all day for one phrase. Nobody typed that program. It was grown from millions of examples, exactly the way we are about to learn.

🧪 TRY IT YOURSELF

Open a voice-recorder app that shows the sound wave. Say your own name twice, one second apart. Look at the two pictures. Same word, same voice, same room — how alike are they? Now imagine writing a rule that matches both.

2

What “learning” really means

You learned to throw a ball before you learned any physics. A machine can learn the same way.

∞ THINK OF IT LIKE... THROWING A PAPER BALL INTO A BIN

You crumple a page and throw it at the bin across the room. Too short. You throw harder. Too far. A little softer... in. After ten throws you rarely miss.

Nobody gave you an equation. You used only three things: you **tried**, you **saw how wrong you were**, and you **adjusted**. Then you repeated. That loop is learning.

So here is the simplest possible definition, the one a child would accept:

IN ONE LINE Learning is getting better at something by practising on examples.

And the machine version is the same sentence with two words changed:

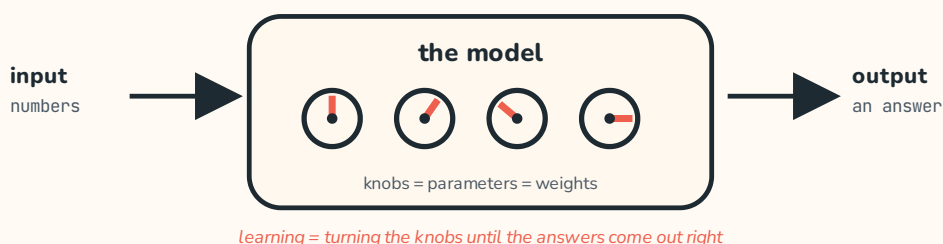
IN ONE LINE Machine learning is a program that gets better at a task by practising on **data**, instead of being given the rules.

The knobs

Here is a question that unlocks everything. *To get better, what must a learner have?*

It must have something it can **change**. If nothing can change, nothing can improve. When you throw, the thing that changes is the strength of your arm. In a program, the things that change are some **numbers inside the program** that we are allowed to adjust.

We will call them **knobs**. Their official name is **parameters**, or **weights**. Same thing. A learning program is a machine with knobs: numbers go in, the knobs do some maths, numbers come out.



A learning program is a machine with knobs. The knobs are numbers. Once they are set well, the machine gives good answers. The final knob settings are the program we could not write by hand.

∞ THINK OF IT LIKE... TUNING AN OLD RADIO

Turn the knob a little. More hiss? Turn back. Clearer voice? Keep going that way. Stop when it is clean. You are the one turning; the hiss tells you how wrong you are. Remember this picture — in Chapter 5 the “hiss” will get a proper name.

Let's train a model by hand

We will do the whole thing — a real machine-learning problem — with one knob, on paper. Asha's college canteen sells chocolate bars. She notes three of them:

bar	weight	price
A	50 g	₹40
B	100 g	₹80
C	25 g	₹20

She wants a program that guesses the price of any bar from its weight. Her model has **one knob**:

$$\text{price} = k \times \text{weight}$$

The knob is “rupees per gram”. Nobody tells Asha its value. She has to find it from the three examples.

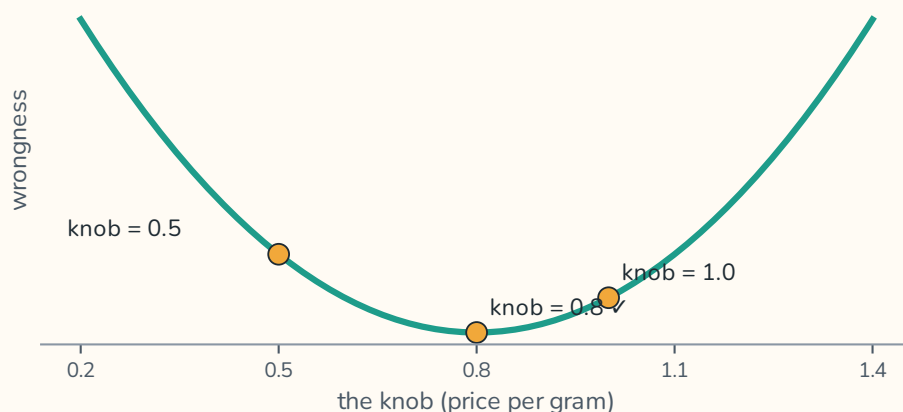
try	knob	guess for A (50 g)	guess for B (100 g)	guess for C (25 g)	verdict
1	1.0	50 (truth 40, off by 10)	100 (off by 20)	25 (off by 5)	too high everywhere → turn the knob down
2	0.5	25 (off by 15)	50 (off by 30)	12.5 (off by 7.5)	too low → turn it up , but less than before
3	0.8	40 ✓	80 ✓	20 ✓	all correct → stop

A peek at how a computer writes try 3. It does not guess one bar at a time. It stacks the three weights in a column and multiplies the whole column by the knob in one go:

$$\begin{bmatrix} 50 \\ 100 \\ 25 \end{bmatrix} \times 0.8 = \begin{bmatrix} 40 \\ 80 \\ 20 \end{bmatrix} \quad \text{and the gaps are} \quad \begin{bmatrix} 40 \\ 80 \\ 20 \end{bmatrix} - \begin{bmatrix} 40 \\ 80 \\ 20 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

three bars or three million, it is still one multiplication and one subtraction. Every book from here on shows its maths this way: first one example at a time, then the whole table at once.

Congratulations. You have just trained a model. Look at what you did: you **guessed** a knob value, you **measured** how wrong the guesses were, you **turned** the knob the right way, and you **repeated**. Now Asha can price a bar she has never seen: a 70 g bar should cost about $0.8 \times 70 = ₹56$. That is a **prediction**.



The wrongness makes a bowl. If we plot “how wrong” for every possible knob value, we get a bowl. The best knob is at the bottom. In Book 3 we will learn how a computer finds the bottom without trying every value — even when there are a million knobs.

💡 STOP AND THINK

The canteen adds two new bars: a 70 g bar for ₹60, and a fancy imported 30 g bar for ₹45. Can Asha's one-knob model get all five prices right? Try a few knob values.

✓ OUR ANSWER

No. With $k = 0.8$ the fancy bar comes out as $0.8 \times 30 = ₹24$, not ₹45. No single “rupees per gram” fits a world where fancy bars cost more. To do better, the model needs to know more (a second input: *is it imported?*) and needs more knobs to use that information.

This is the first law of modelling: **the model must be rich enough for the world it lives in**. Deep networks have millions of knobs for exactly this reason. And millions of knobs cannot be turned by hand — so the turning itself must be done by an algorithm. That algorithm is the hero of Book 3.

🔧 TRY IT YOURSELF — THE TWO-KNOB GAME

The canteen changes its pricing: every bar now includes a ₹5 wrapper charge. The new data: **A** 50 g → ₹45, **B** 100 g → ₹85, **C** 25 g → ₹25. Asha gives her model a second knob:

$$\text{price} = k_1 \times \text{weight} + k_2$$

k_2 is added on its own, not multiplied by anything. (In Chapter 5 you will learn its name: the **bias**.) Train the model by hand. Row 1 is done; fill in the rest until all three guesses are right.

try	k_1	k_2	guess A (50 g) — truth 45	guess B (100 g) — truth 85	guess C (25 g) — truth 25	verdict
1	1.0	0	50 (too high by 5)	100 (too high by 15)	25 ✓	big bars too high → lower knob ₁
2						
3						

Hint: compare B with A. Fifty extra grams cost ₹40 extra — that tells you knob₁ on its own. Then use any one bar to find knob₂.

Notice what changed: with two knobs you had to be clever about which one to fix first. With a million knobs, nobody can do this by hand. Book 3 gives the method that can.

answer: knob₁ = 0.8, knob₂ = 5 → 45 ✓ 85 ✓ 25 ✓

📖 WORDS TO KNOW

Parameter / weight — a knob: a number inside the model that learning is allowed to change. **Model** — the machine with knobs. **Training** — turning the knobs using examples. **Prediction** — the model's answer for a new input.

So what is “deep” learning?

Deep does not mean deep thoughts. It means many layers, one on top of the other.

In Chapter 2, our model was one knob and one multiplication. It could learn a price per gram and nothing more. Nova’s job — 16,000 numbers in, yes/no out — needs something far richer. Here is the idea that makes it possible.

💡 BIG IDEA — THE DEFINITION

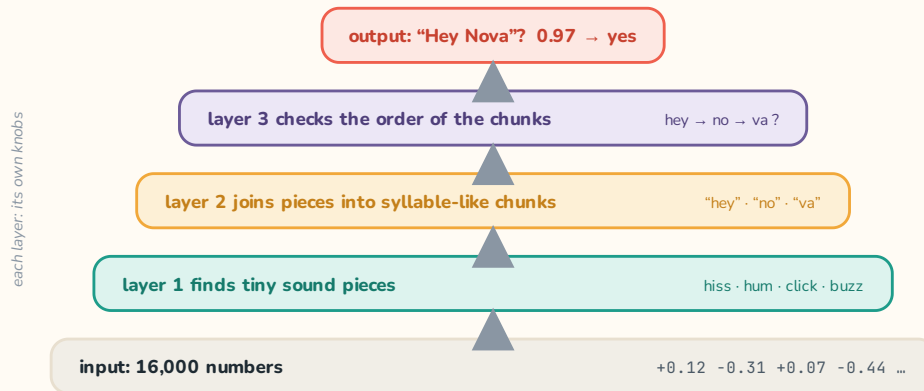
Deep learning is machine learning where the model is a **stack of many simple layers**. Each layer takes what the layer below gives it and turns it into something a little more useful. All the layers are learned *together*, from examples.

The word *deep* just counts the layers. Two layers: shallow. Twenty layers: deep. Nothing more mysterious than that.

🔗 THINK OF IT LIKE... READING A HANDWRITTEN WORD

Your eye receives dots of light. Something in you turns dots into strokes. Something turns strokes into letters. Something turns letters into a word. Each step is small and easy. *No single step knows the word.* But the stack of steps does.

That is exactly how a deep network handles Nova’s sound.



Nova’s stack. Each layer does one small, learnable job and passes its result upward. Nobody told layer 1 to look for hisses and hums. It *learned* that those pieces are useful for the layers above it. (The exact jobs shown here are a simplification, but the spirit is right.)

🔗 THINK OF IT LIKE... AN ASSEMBLY LINE

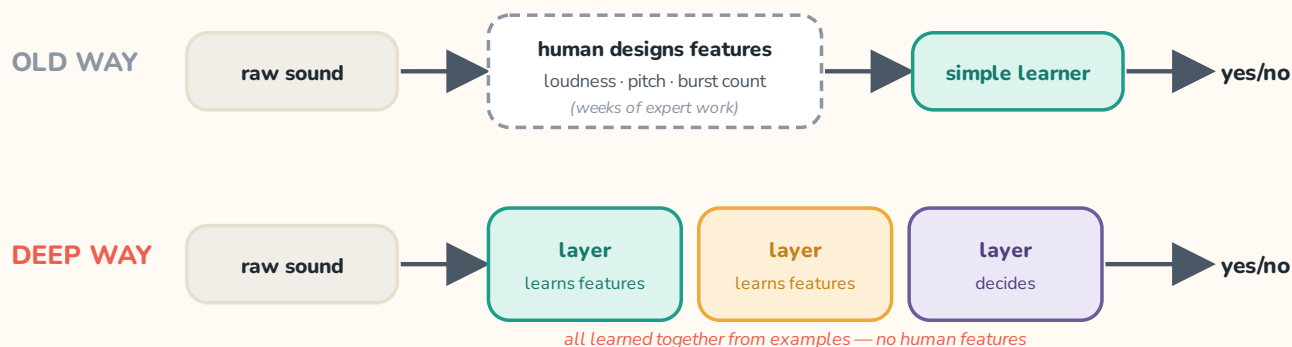
In a car factory, nobody at any single station can build a car. One station fits doors; another paints; another checks the lights. Each job is simple. The *line* builds cars. A deep network is an assembly line for answers.

Old way, new way

Machine learning existed long before deep learning. What changed?

In the old way, a human expert designed the **measurements** to take from the raw data — “measure the loudness”, “measure the pitch”, “count the bursts”. These measurements are called **features**. Then a simple learner, with just one layer of knobs, used those features to decide. The learner was small; the cleverness was in the human.

In deep learning, the raw numbers go straight in. The layers **learn their own measurements**. This is the real magic: the machine invents features that we could never have written down — because, as Chapter 1 showed, we do not know them ourselves.



Where the cleverness lives. Old way: a human invents the features, a small learner decides. Deep way: the layers invent the features themselves. The human’s job moves from “write the rules” to “collect good examples and design the stack”.

⚠ CAREFUL — A LAYER IS NOT MAGIC

In Book 5 you will see that one layer is just this: *multiply the inputs by knobs, add them up, then squash the result*. That’s all. Depth means repeating this simple move many times. Simple steps, stacked, become powerful. Keep that in mind whenever deep learning sounds like sorcery.

✨ SPARK

Your own visual system works in stages. The first stage of your visual cortex responds to tiny edges and lines; later stages respond to shapes; later still, to faces and objects. When scientists looked inside deep image networks, they found the same story: early layers detect edges, later layers detect eyes and wheels. Nobody programmed that. Both learned it.

📖 WORDS TO KNOW

Layer — one stage of the stack; has its own knobs. **Feature** — a useful measurement of the input (learned, in deep learning).

Representation — what a layer passes upward: the input, re-described more usefully.

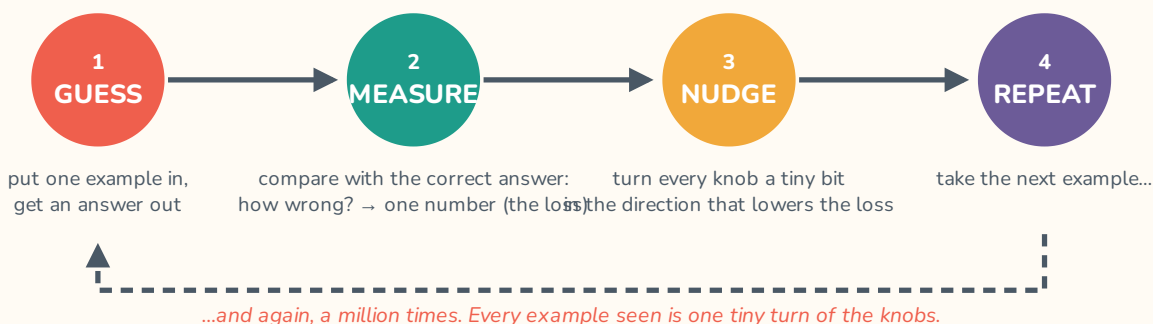
Neural network — a model built from layers of tiny units called *neurons* (Book 2). **Depth** — the number of layers.

4

How it works: the learning loop

Guess. Measure. Nudge. Repeat. That is the whole engine — for one knob or for a billion.

In Chapter 2 you trained a one-knob model by hand. Deep learning does the very same four moves — with millions of knobs, done by a program, millions of times. Here is the loop.



The learning loop. Nothing clever happens in any single turn of the knobs. The cleverness is in doing it a million times.

1 • Guess

Put one example in — one clip, 16,000 numbers. The knobs do their maths. An answer comes out, say “0.61 sure it was Hey Nova”. At the start the knobs are random, so the first guesses are nonsense. That is fine. Everyone starts by missing the bin.

2 • Measure

Compare the guess with the label (yes, it was Hey Nova). Turn the gap into **one number**: how wrong. Big number, very wrong. Zero, perfect. This number is the **loss**.

3 • Nudge

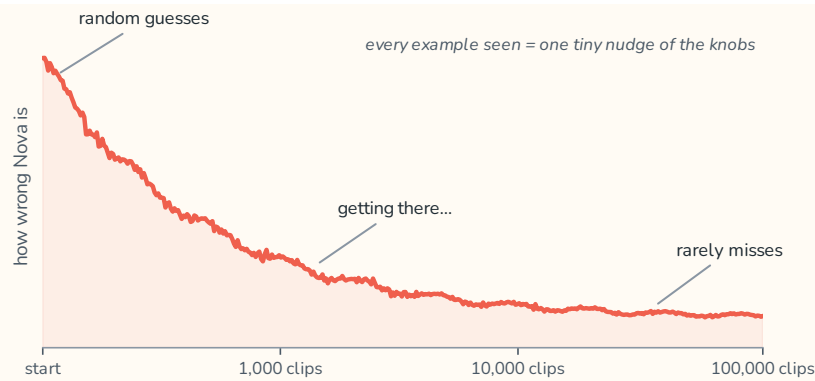
Move every knob a tiny bit in the direction that would have made the loss smaller. Not a jump — a nudge. (How the program knows the right direction for a million knobs at once is the star of Books 3 and 5. For now: it does.)

4 • Repeat

Take the next example. Guess, measure, nudge. Go through all the examples, then again, and again. Each full pass through the data is called an **epoch**.

🔗 THINK OF IT LIKE... A SHOWER TAP

You never jump straight to the perfect temperature. Too cold — turn a little towards hot. Too hot — turn back a little. Check, turn, check, turn. You creep towards “just right” with small moves, using the water on your hand as the loss. That is the learning loop, with you as the optimizer.



What learning looks like from the outside. As Nova sees more clips, the loss falls. At first it drops fast (easy lessons), then slowly (hard cases). A plot like this is the first thing every deep learning engineer looks at. You will draw a real one from your own code in Book 3.

⚠ CAREFUL — NOVA DOES NOT “UNDERSTAND”

After training, Nova has knob settings that give the right answer on the examples. It has not learned what a name is, or who Asha is. A strange new sound — a song with the same rhythm — can fool it. Deep learning is **pattern-finding**, not understanding. Holding on to this honesty will make you a better engineer than most.

Two phases: learning, then using

The loop is only half the story. Once the knobs are set, we **freeze** them and simply use the model. These are two very different phases, and mixing them up causes a lot of confusion.

	Training (learning)	Inference (using)
like	studying for the exam with solved questions	writing the exam
needs	examples <i>with</i> correct answers	only the input; no answer key
knobs	move on every example	frozen
speed	slow: hours or days, big computers	fast: milliseconds, can run on a tiny chip
happens	once, in the lab	every time someone speaks near Nova

Nova on your desk is in the second phase: its knobs were set long ago, and it only runs the “guess” step now — and calls the guess a **prediction**.

💡 STOP AND THINK

Why *tiny* nudges? If Rohan’s clip shows the knobs are wrong, why not fix them completely, in one big jump?

✓ OUR ANSWER

Because one example is a *hint*, not the truth. If Rohan’s clip moved the knobs a lot, Nova would become a Rohan-detector and forget Asha. Small steps let thousands of hints average out into a rule that works for everyone. (There is a second reason — a big jump can leap right over the bottom of the bowl and land higher on the other side. Book 3 shows this with actual numbers.)

5

The four ingredients

Every deep learning project — every single one — is made of the same four things. Learn to spot them and nothing will look strange again.

Self-driving cars, chatbots, Nova, a model that predicts rain — under the surface they are all built from the same four ingredients. Let's meet them first through a picture you already understand: learning archery with a coach.

∞ THINK OF IT LIKE... LEARNING ARCHERY

You shoot an arrow. It lands to the left of the bullseye. Your coach says “aim a bit right”. You shoot again. Closer. Again. Four things are at work here, and they map exactly onto deep learning.

● Data

Archery: every practice shot, together with where it landed.

Deep learning: examples — each one an input plus the correct answer.

● Model

Archery: you — your stance, your pull, your aim. The adjustable things.

Deep learning: the machine with knobs.

● Objective function

Archery: the score — distance from the bullseye. One number. Smaller is better.

Deep learning: the loss — one number for “how wrong”.

● Optimization algorithm

Archery: the coach's rule: “it went left, so aim a little right.”

Deep learning: the recipe that turns the loss into knob adjustments.

Now each ingredient, properly.

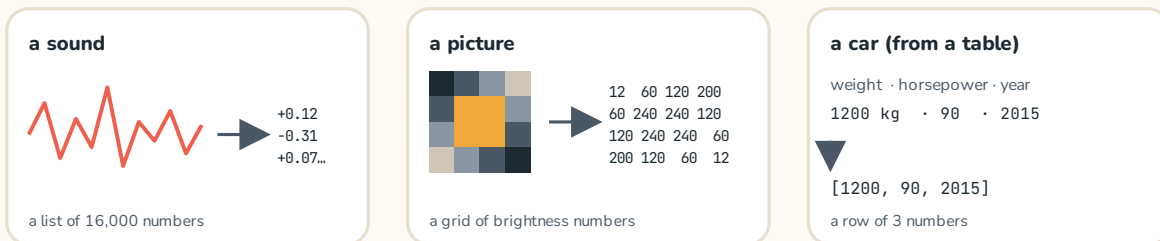
Ingredient 1 Data — the experience

Data is a big pile of **examples**. One example is two things: an **input** and the **correct answer**. The correct answer is called the **label**.

For Nova: the input is a one-second clip (16,000 numbers). The label is 1 if the clip contains “Hey Nova” and 0 if it does not. Asha needs thousands of both kinds.

Everything becomes numbers

A model can only do maths, so every kind of input must first be turned into numbers. Sound becomes a list. A picture becomes a grid of brightness values. A car in a spreadsheet becomes a row of measurements. A list of numbers has a name — a **vector** — and the whole world of deep learning is vectors going in and vectors coming out.



Everything becomes a vector. Sound, pictures, table rows — all end up as lists of numbers before a model sees them. Book 3 uses table rows (cars). Book 4 uses table rows (tumours, flowers). The idea is the same for sound and images.

Data must be many, and must be varied

A model can only learn what the data shows it. Two rules follow.

- **Many.** Three chocolate bars were enough for one knob. Millions of knobs need millions of examples. Deep learning is hungry.
- **Varied.** If every “Hey Nova” clip comes from men in quiet rooms, Nova will fail for women and on noisy streets. It is not being unfair on purpose; it simply never saw those cases. Data that is missing something teaches a model that is missing something. Engineers say: *garbage in, garbage out*.

💡 BIG IDEA — HIDE SOME DATA

Never let the model see *all* the examples during training. Keep some aside — the **test set** — and never train on them. Afterwards, ask the model about the hidden examples. If it does well on clips it has never heard, it has truly learned. If it only does well on the clips it trained on, it has merely **memorised**.

You know this trick already: you practise on past papers, but the exam has new questions. A student who only memorised the past papers is found out on exam day.

Ingredient 2 Model — the machine with knobs

The model is the machine from Chapter 2: numbers in, maths with knobs, numbers out. Choosing a model means choosing **how many knobs** there are and **how they are connected**.

The simplest useful model is this: multiply each input number by its own knob, add everything up, and add one extra knob on its own (called the **bias**). That is a **linear model**, and it is exactly one artificial **neuron**. Books 2, 3 and 4 are about this one neuron, and it can do a surprising amount.

$$\text{output} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

the red letters are knobs; the x 's are the inputs. In shorthand, $\text{output} = \mathbf{w} \cdot \mathbf{x} + b$ — and for a whole table of examples stacked as the rows of a matrix $\mathbf{X}$, every output at once is $\mathbf{X}\mathbf{w} + b$. That one line is what a computer actually runs; Book 3 shows it with numbers.

A **deep model** is many of these neurons, arranged in layers, each layer feeding the next (Book 5). More knobs means the model can fit a richer world — remember the fancy chocolate bar. But more knobs also means more data is needed, and a bigger risk of memorising instead of learning.

Ingredient 3 Objective function — the score

A program cannot hear “that was kind of okay”. It needs **one number** that says how wrong a guess was. That number is the **loss**. Lower is better; zero is perfect. The whole of training is one instruction: *make the loss as small as you can*. Mathematicians say “**minimise** the loss”.

Different kinds of answers need different ways of scoring.

When the answer is a number (“how much?”)

Use the **squared error**: take the gap between guess and truth, and square it.

$$\text{loss} = (\text{guess} - \text{truth})^2$$

truth 50: guess 40 $\rightarrow (40 - 50)^2 = 100$;
guess 48 $\rightarrow (48 - 50)^2 = 4$; guess 60 $\rightarrow$
 $(60 - 50)^2 = 100$

Why square? Two reasons. It removes the minus sign, so guessing 10 too low and 10 too high count the same. And it punishes big misses much more than small ones: a miss of 10 costs 100, a miss of 2 costs only 4.

This is the loss of Book 3.

When the answer is a category (“which one?”)

The model gives a **confidence**, like “0.9 sure it is Hey Nova”. The loss asks: *how surprised should we be, given the truth?* If it was Hey Nova and the model said 0.9, small surprise, small loss. If it said 0.1 — big surprise, big loss.

This “surprise” loss has a grand name, **cross-entropy**, but that is the whole idea.

This is the loss of Book 4 — and the loss Nova needs.

You will hear four names for the same thing: *loss function*, *cost function*, *error function*, *objective function*. Do not let the names scare you. They all mean “the score we try to lower”.

Ingredient 4 Optimization algorithm — the knob-turner

We have knobs, and we have a score. The last ingredient is the procedure that turns the knobs to lower the score. By far the most common one is **gradient descent**. Its rule fits in three lines:

💡 GRADIENT DESCENT, IN PLAIN WORDS

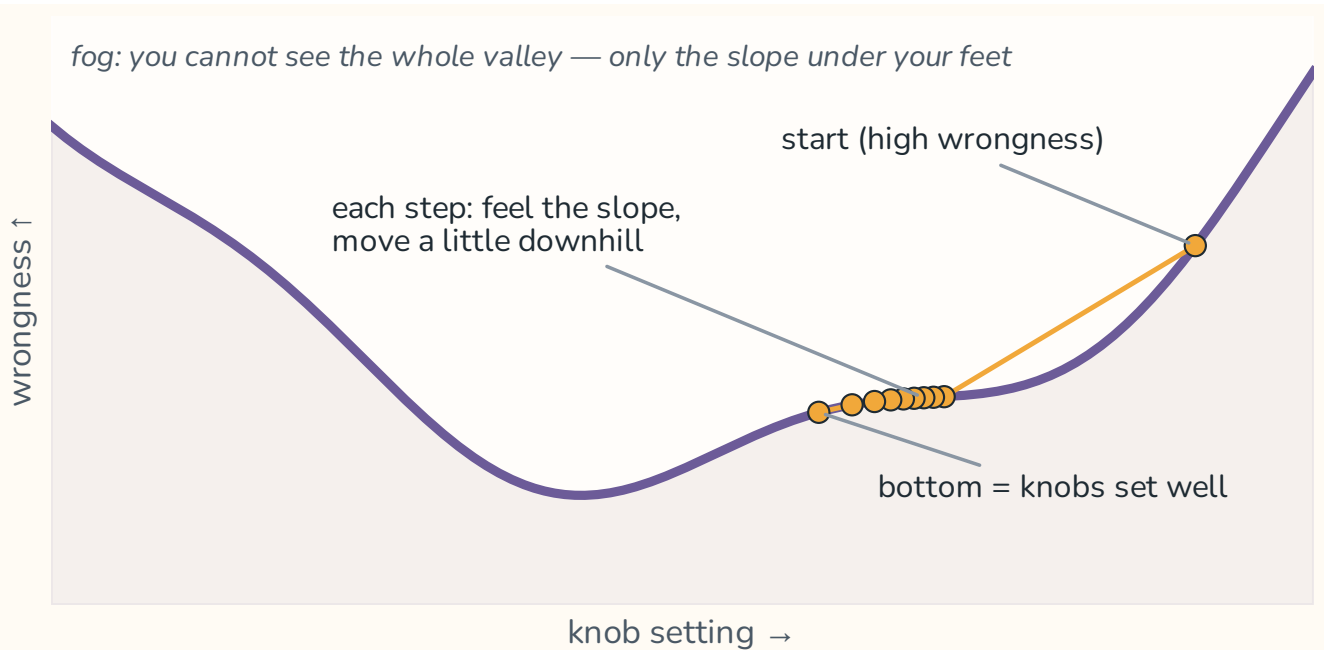
For each knob, ask: “if I turn this knob up a tiny bit, does the loss go up or down?”

Turn the knob the way that makes the loss go **down**. Take a small step.

Repeat.

$$\text{knob} \leftarrow \text{knob} - \eta \times \text{slope}$$

“slope” is the answer to the question (positive: loss goes up); η is the small step size. Book 3 derives the slope and runs this line by hand.



Walking downhill in fog. Wrongness is a landscape over the knob settings. You cannot see the valley, but you can feel the slope under your feet. Step downhill, feel again, step again. The slope has a maths name — the **gradient** — and “going down it” is why the method is called *gradient descent*.

✦ SPARK — A PREVIEW OF THE MAGIC

“Ask, for each knob, whether the loss goes up or down” sounds slow when there are a million knobs. Yet with a little calculus a computer can answer that question for *all* the knobs in a single pass. That trick is called **backpropagation**, and it is the reason deep learning works at all. It is the finale of Book 5. Everything before it is preparation.

The four ingredients, side by side

	Archery	Chocolate price (Ch. 2)	Nova's wake word
Data	shots + where they landed	3 bars: weight → price	thousands of clips → yes / no
Model	your stance and aim	price = knob × weight	a stack of layers with millions of knobs
Objective	distance from bullseye	squared error of the prices	surprise at the true answer
Optimizer	the coach's “aim a bit right”	you, turning the knob by hand	gradient descent, a million times

IN ONE LINE Data gives experience. The **model** gives knobs. The **objective** gives a score. The **optimizer** turns the knobs to lower the score.

Worksheet: spot the four ingredients

Pick “smart” features you use every day. For each one, fill in the four ingredients. The first row is done for you. (Hint for Maps: the label is how long the trip *actually* took — the app learns it when you arrive, so every trip you make is one more example.) If you can fill three rows without help, you have understood this chapter better than most people who use these products.

Product	Data (examples + labels)	Model (what has knobs?)	Objective (what is scored?)	Optimizer (who turns the knobs?)
Spam filter	millions of emails, each marked spam / not spam by users	a network reading the words of an email → “how sure it is spam”	surprise at the true label (cross-entropy)	gradient descent, run by the email company
Face unlock				
Maps: “arriving in 14 min”				
Song recommendations				
Your choice: _____				

6

Why deep learning? And why now?

Machine learning is old. Layers are old. So why did the world change in the last fifteen years?

Four honest reasons to use it

1 • It works where rules fail

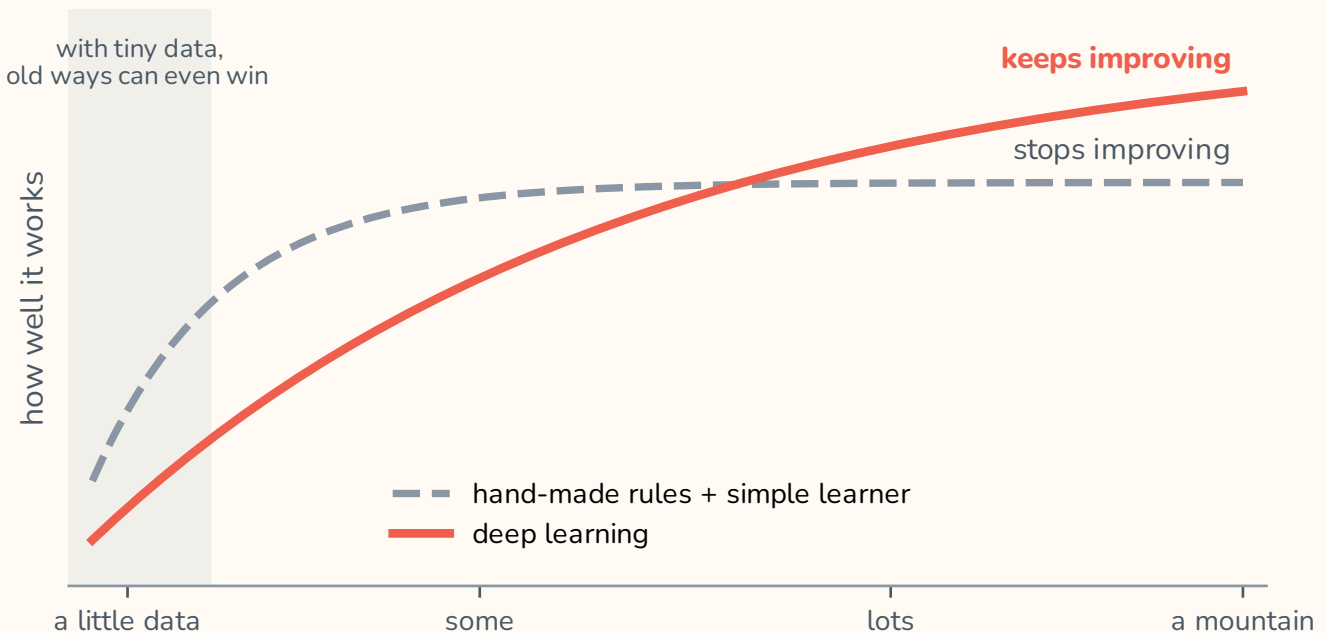
Seeing, hearing, reading. These are exactly the tasks where we know more than we can tell (Chapter 1). For fifty years, computers were bad at them. Deep learning made them good — because it learns the features we could never write.

✦ SPARK — THE MOMENT THINGS CHANGED

In 2012, a yearly contest asked programs to name the object in a photo, choosing from a thousand categories. For years the best hand-designed systems got about one in four photos wrong. That year a deep network from a small team in Toronto got about one in six wrong — a gap so large that many researchers switched fields within months. Almost every image system you use today descends from that moment.

2 • It keeps getting better with more data

Old methods hit a ceiling. Once the human-designed features have said everything they can say, more data does not help. Deep models have no such ceiling, because more data lets the layers discover richer features. The picture below is the single most important reason companies bet on deep learning: they have mountains of data, and this is the method that uses it.



Data is fuel. With little data, old methods may even win. Add more data and they flatten out; deep learning keeps climbing. (A sketch of the general pattern, not a specific measurement.)

3 • One recipe, many problems

The four ingredients do not care what the numbers mean. Swap the sound clips for photos, or sentences, or protein shapes, or chess positions — and the same recipe works. Learn it once for Nova and you have learnt it for everything.

4 · Why now: three things arrived together

Data

Phones, cameras and the internet started producing more labelled examples in a day than existed in all of history before. Hungry models finally had food.

Computing

Chips built for video games (GPUs) turned out to be perfect for the maths of layers, which is mostly multiplying big grids of numbers. Training that took months now takes hours.

Open tools

Free libraries and shared research meant a student anywhere could train a serious model. The ideas spread at the speed of a download.

CAREFUL — IT IS NOT MAGIC

Deep learning needs lots of labelled data and lots of computing. It is hard to explain *why* it decided something. It learns the mistakes and biases hiding in its data. And for small, clean problems, a simple model is often better, faster and easier to trust. Knowing when *not* to use deep learning is part of being good at it.

STOP AND THINK

Asha has recorded only 30 clips of “Hey Nova”. Should she start training a deep network today?

OUR ANSWER

Probably not yet. Thirty examples cannot teach a million knobs; the model would memorise all thirty and fail on the thirty-first. Asha’s best move is to collect more clips — many voices, many rooms — or to begin with a much smaller model. Of the four ingredients, data is the one that most decides success. Spend your effort there first.

TRY IT YOURSELF

Over one day, note three things around you that almost certainly use deep learning (a camera that finds faces, a keyboard that predicts your next word, a video app that adds subtitles). Then find one thing where a simple rule would clearly be the better choice (a lift that goes to the floor you pressed). Write one line on why each choice is right. Knowing the difference is a skill employers actually pay for.

7

Supervised learning: learning with an answer key

Everything in this course is one kind of learning. Here is what that kind is, and the questions it can answer.

IN ONE LINE Supervised learning is learning from examples that come **with the correct answer**. Every example is a question and its answer key.

The “supervisor” is not a person standing over the machine. The supervisor is the **label** — the correct answer attached to each example. It is like practising with a solved question bank: you attempt a question, look at the answer, see how far off you were, and adjust. Nova’s clips have labels (yes / no). The chocolate bars had labels (their prices). Both are supervised learning.

Different questions need different kinds of answers. Here are the ones you will meet most often.



Regression

“How much? How many?”

The answer is a **number**. Car’s weight and engine size → its fuel mileage. House size → price. Today’s weather → tomorrow’s temperature. Order details → delivery time.

Book 3 one neuron predicts the mileage of real cars.



Binary classification

“Yes or no? Which of two?”

The answer is **one of two categories**. Is this email spam? Is this tumour harmful? Will this customer leave? Does this clip contain “Hey Nova”?

Book 4 one neuron looks at real tumour measurements and answers harmful / harmless.



Multi-class classification

“Which one of several?”

The answer is **one category out of many**. Which of three flower species is this? Which digit, 0 to 9, is this handwriting? Which of a thousand objects is in this photo? Which word did the speaker say?

Book 4 a row of neurons sorts real flowers into three species.



Tagging

“Which ones apply?”

The answer is **several yes/no’s at once**. A photo may contain a cat *and* a dog *and* grass. A news article may be about sport *and* money. Each tag is its own little yes/no question.

hello



Sequence learning

“Turn this sequence into that one.”

Input and output are both **sequences**, often of different lengths. Speech → written text. English → Hindi. A question → an answer, word by word (this is what chatbots do).

- 1 
- 2 
- 3 

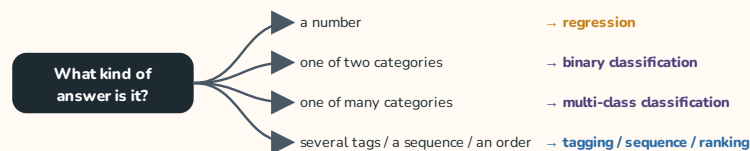
Ranking & recommendation

“Which is best? What would this person like?”

The answer is an **ordering** or a **score for each option**. A search engine orders pages for your query. A music app scores every song by how much you would like it and shows the top few.

How to tell them apart: look at the output

Students often ask “is this regression or classification?” There is a one-step test: ignore the input and look only at the kind of answer.



The one-step test. The input can be anything — sound, pixels, a table row. Only the shape of the *answer* decides the kind of task, the kind of loss, and the kind of output neuron.

🔔 STOP AND THINK — SORT THESE EIGHT

For each task, name the kind: regression, binary classification, multi-class classification, tagging, sequence learning, or ranking. Use only the shape of the answer.

- (a) Nova’s wake word. (b) The chocolate price from Chapter 2. (c) Which of the 60 students in the class wrote this handwriting? (d) Predict the final score of a cricket innings from the first ten overs. (e) Decide whether a bank transaction is fraud. (f) Name the raga of a song, out of ten ragas. (g) Write subtitles for a video. (h) Choose which five videos to show you next.

✓ OUR ANSWER

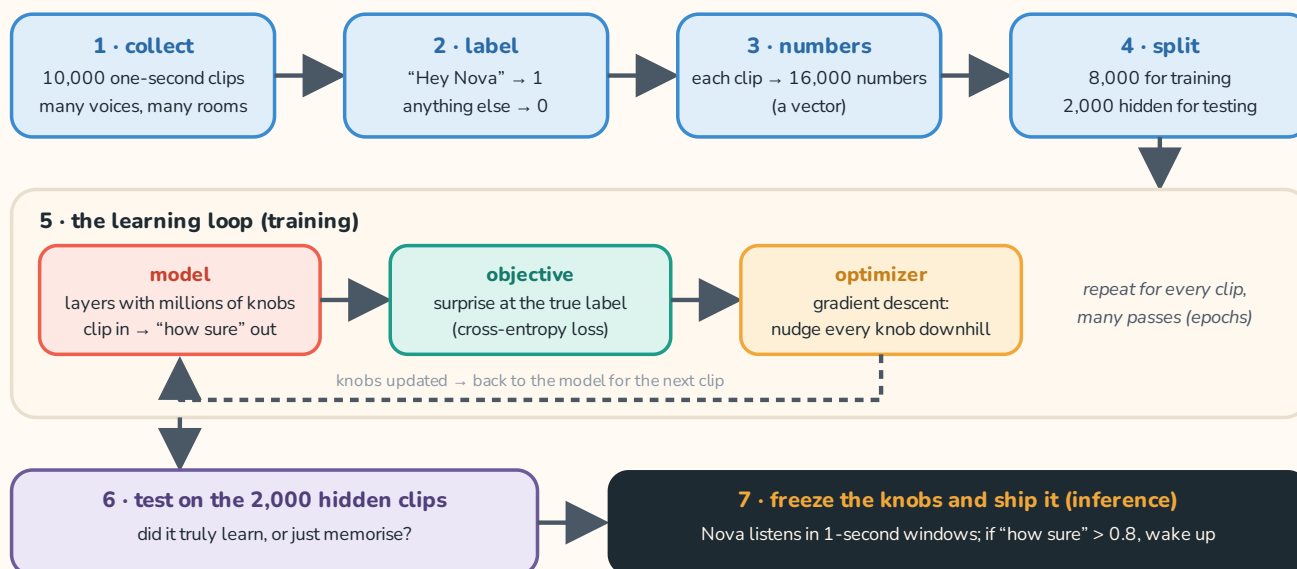
- (a) yes/no → **binary classification**. (b) a price → **regression**. (c) one of 60 → **multi-class classification**. (d) a score → **regression**. (e) yes/no → **binary classification**. (f) one of ten → **multi-class classification**. (g) sound → text → **sequence learning**. (h) an ordering → **ranking**. Notice that you never needed to look at the inputs.

Not everything has an answer key

Two other kinds of learning exist, and you should know their names. In **unsupervised learning** there are no labels at all — the model is given a pile of photos and asked to find natural groups, like sorting your camera roll into “beach”, “friends” and “food” without being told those words. In **reinforcement learning** the model learns from *rewards* rather than answers — like training a puppy with treats. This course lives in the supervised world; the four ingredients appear in the other two as well, in slightly changed clothes.

Nova's recipe, start to finish

Let's put every piece from this book in one line, and watch the whole thing run.



The whole recipe. Steps 1–4 are the data ingredient. Step 5 is the model, the objective and the optimizer running the learning loop. Step 6 is the honesty check. Step 7 is Nova, finally awake when it should be.

STORY — THREE WEEKS LATER

Asha: Hey Nova.

Nova: *(eyes light up)*

Rohan: Hey Nova!

Nova: *(eyes light up)*

Rohan: Hey Nora.

Nova: *(nothing)*

Rohan: How did you write the rule for that?

Asha: I didn't. I collected ten thousand clips, and the rule wrote itself.

Where the rest of this course goes

Look again at step 5. Inside "layers with millions of knobs" and "nudge every knob downhill" sit all the ideas of the next four books. We will build them by hand, from nothing: **Book 2** finds one artificial neuron inside the model and teaches it logic puzzles (and meets one it cannot learn). **Book 3** gives one neuron a real regression job, the squared-error loss, and gradient descent with the maths shown step by step. **Book 4** gives one neuron a real yes/no job with the surprise loss, then a row of neurons a "which of three" job. **Book 5** stacks neurons into layers, derives backpropagation, and closes the loop back to Nova. Every book uses the same four ingredients and the same loop you now know. Nothing new will be added to the *shape* of the idea — we will only make each piece real.



Look back, look ahead

The whole book in six sentences, a self-check, some questions worth carrying, and a glimpse of Book 2.

The story in six sentences

1. Some programs cannot be written by hand, because **we do not know the rule ourselves** — hearing a name is one of them.
2. So we **show instead of tell**: collect examples and let a machine with **knobs** find the rule. That is machine learning.
3. **Deep learning** is machine learning where the machine is a **stack of simple layers** that learn their own features, all together.
4. It learns by a loop: **guess, measure, nudge, repeat** — millions of tiny turns of the knobs.
5. Every project has **four ingredients**: data (experience), model (knobs), objective (score), optimizer (knob-turner).
6. We work in **supervised learning** — examples with answer keys — and the *shape of the answer* tells us the kind of task: a number (regression), or a category (classification).

Explain it to a friend

Cover the right column. Answer out loud, in your own words. Then compare.

Question	A good short answer
Why can't we just write the wake-word program?	Because the input is 16,000 raw numbers that change with every voice and room, and we don't know the rule ourselves — we know more than we can tell.
What is a parameter?	A knob: a number inside the model that training is allowed to change. The trained knob values <i>are</i> the program.
What does “deep” mean?	Many layers. Each layer re-describes its input a bit more usefully; all layers are learned together.
What is a loss?	One number that says how wrong a guess was. Lower is better. Training tries to make it as small as possible.
Why hide a test set?	To check the model truly learned rather than memorised — like an exam with new questions after practising on past papers.
Regression or classification?	Look at the answer. A number → regression. A category → classification.
What does gradient descent do?	For each knob, finds which way lowers the loss and takes a small step that way. Repeats. Walking downhill in fog.

✦ QUESTIONS WORTH CARRYING (NO ANSWERS HERE — ON PURPOSE)

- 1 · If Nova only found knob settings that fit the examples, does it *know* its name? Where exactly is the line between finding patterns and understanding?
- 2 · You learned to catch a ball with no equations. Is your brain also “turning knobs”? What would the knobs be?
- 3 · What data would you have to collect for Nova to work for every language spoken in India? What would go wrong if you collected it only on campus?
- 4 · Chapter 2 needed a second input (“is it imported?”) to price fancy chocolate. For the wake word, we never told the model what to measure. Why can it get away with that, when the chocolate model could not?

Words to know — the whole book

Machine learning

A program that improves at a task by using data, instead of being given the rules.

Deep learning

Machine learning where the model is a stack of many layers, all learned together.

Model

The machine with knobs: numbers in, maths, numbers out.

Parameter / weight

A knob — a number inside the model that training changes.

Bias

One extra knob added on its own, not multiplied by any input.

Layer

One stage of a deep model, with its own knobs.

Feature

A useful measurement of the input. Hand-designed in old ML; learned in deep learning.

Example

One input together with its correct answer.

Label

The correct answer attached to an example.

Vector

A list of numbers. Every input becomes one.

Loss / objective

One number for “how wrong”. Lower is better.

Squared error

$(\text{guess} - \text{truth})^2$, the loss for number answers.

Cross-entropy

The “surprise” loss for category answers.

Optimizer

The procedure that turns knobs to lower the loss.

Gradient descent

The optimizer that steps each knob a little downhill, repeatedly.

Training

Running the learning loop to set the knobs.

Inference / prediction

Using the frozen model on new inputs.

Epoch

One full pass through all the training examples.

Test set

Examples hidden during training, used to check real learning.

Supervised learning

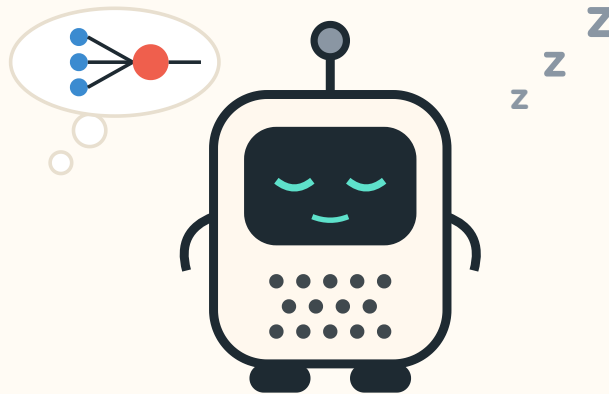
Learning from examples that come with correct answers.

Regression

A task whose answer is a number.

Classification

A task whose answer is a category (binary: two; multi-class: many).



End of Book 1

Nova is asleep. Somewhere inside it, a single neuron is waiting to be found.

NEXT TIME — BOOK 2: ONE TINY BRAIN CELL

We open Nova's head. Inside the layers, the smallest working part is a single **artificial neuron**: a few inputs, a few knobs, a sum, and a decision. We will compare it with the real neurons in your brain, and then teach it — by hand, with pencil and paper — to behave like the logic gates AND, OR and NOT. It will feel almost too easy.

Then we will hand it one more puzzle, called XOR. It has only four examples. And the neuron will fail, no matter how long we train it. Finding out *why* it fails — and what it takes to fix it — will carry us all the way to Book 5.

Deep Learning, Told Simply · a five-book story for the Deep Learning (DNN) course · Prof. Saurabh